

ShopWiser Matching Pipeline

March 3, 2026

1 Cross-Retailer Product Identity Matching

1.1 Problem Statement

We are given product catalogues from multiple grocery retailers (e.g., Tesco, Sainsbury’s, Waitrose). Each retailer provides a set of items (rows) with fields such as title text, category, brand indicators, and pricing information. We assume there is *no shared global identifier* (e.g., GTIN/EAN) across retailers.

Let $\mathcal{S}$ denote the set of retailers. For each retailer $s \in \mathcal{S}$, we have a set of items $\mathcal{I}_s$. Each item $i \in \mathcal{I}_s$ has metadata x_i (text, category, brand flags, price information, etc.).

Goal. For each ordered retailer pair (s_a, s_b) with $s_a \neq s_b$, we wish to learn a function

$$f_\theta(i, j) \in [0, 1] \quad \text{for } i \in \mathcal{I}_{s_a}, j \in \mathcal{I}_{s_b},$$

that outputs the probability that (i, j) correspond to the **same commercial product identity** (“exact SKU identity”) across retailers.

Output. A set of cross-retailer match edges

$$\mathcal{E} = \{(i, j) : f_\theta(i, j) \geq \tau_{\text{accept}}\}.$$

If no candidate match exceeds confidence thresholds, we return “no match” for that retailer pairing.

Scope note (own-brand). In strict *identity* matching mode, we typically *exclude* cross-retailer matching when either item is “own brand” because own-brand products are generally not identical across retailers (they are closer to *substitutions* than identities). This can be revisited if the product spec expands beyond identity to substitution equivalence.

1.2 Computed Features

Assume each item i has raw fields such as:

`titlei`, `cleaned_titlei`, `categoryi`, `brandi`, `own_brandi`, `pricei`, `unit_pricei`, `uniti`.

We compute canonicalized and derived features as follows.

1.2.1 Text Canonicalization

```
title_normi := canonicalize(cleaned_titlei)
brand_normi := canonicalize(brandi)
category_normi := canonicalize(categoryi)
```

where `canonicalize` includes lowercasing, punctuation/whitespace normalization, and optionally abbreviation expansion.

1.2.2 Implied Package Size from Pricing

When `unit_price` is in £/unit and `price` is in £, we can infer net quantity. Define a base-unit size `sizei`:

$$\text{size}_i := \begin{cases} 1000 \cdot \frac{\text{price}_i}{\text{unit_price}_i} & \text{if } \text{unit}_i = \text{kg} \quad (\text{grams}) \\ 1000 \cdot \frac{\text{price}_i}{\text{unit_price}_i} & \text{if } \text{unit}_i = \text{l} \quad (\text{millilitres}) \\ \frac{\text{price}_i}{\text{unit_price}_i} & \text{if } \text{unit}_i = \text{unit} \quad (\text{count}) \\ \text{missing} & \text{otherwise.} \end{cases}$$

We also define a coarse bucket (for gating and/or embedding text enrichment):

```
size_bucketi := bucketize(sizei),
```

e.g., rounding to nearest 5–10 (g/ml) (if deemed necessary).

1.2.3 Pairwise Features for a Candidate Pair (i, j)

For any cross-retailer candidate pair (i, j) we compute:

Numeric features.

$$\begin{aligned} \Delta_{\text{size}}(i, j) &:= \frac{|\text{size}_i - \text{size}_j|}{\max(\text{size}_i, \text{size}_j)} \quad (\text{when both sizes exist}) \\ 1_{\text{unit}}(i, j) &:= 1[\text{unit}_i = \text{unit}_j] \\ \Delta_{\text{price}}(i, j) &:= \frac{|\text{price}_i - \text{price}_j|}{\max(\text{price}_i, \text{price}_j)} \quad (\text{optional; not always reliable}) \end{aligned}$$

Metadata agreement features.

$$\begin{aligned} 1_{\text{brand}}(i, j) &:= 1[\text{brand_norm}_i = \text{brand_norm}_j] \quad (\text{when both non-empty}) \\ 1_{\text{cat}}(i, j) &:= 1[\text{category_norm}_i = \text{category_norm}_j] \quad (\text{or same coarse category}) \\ 1_{\text{own}}(i, j) &:= 1[\text{own_brand}_i \vee \text{own_brand}_j] \end{aligned}$$

Text similarity features (examples).

$$\begin{aligned} \text{jac}(i, j) &:= \text{token Jaccard}(\text{title_norm}_i, \text{title_norm}_j) \\ \text{tri}(i, j) &:= \text{char trigram cosine}(\text{title_norm}_i, \text{title_norm}_j) \end{aligned}$$

Embedding features. Let $e_i, e_j \in \mathbb{R}^d$ be learned text embeddings (defined below). Then:

$$\text{dot}(i, j) := e_i^\top e_j \quad (\text{or cosine similarity if vectors are normalized}).$$

2 Training Pipeline.

Below defines the pipeline for training the desired f_θ .

2.1 Level A: Candidate Generation (High Recall)

Goal. Avoid $O(|\mathcal{I}_{s_a}| |\mathcal{I}_{s_b}|)$ comparisons by retrieving a small candidate set per item.

2.1.1 A1. Construct embedding text and compute vectors

For each item i , form an embedding string that optionally includes weakly structured tokens:

$$\text{embed_text}_i := \text{brand_norm}_i \parallel \text{title_norm}_i \parallel \text{category_norm}_i \parallel \text{size_bucket}_i.$$

Compute embedding:

$$e_i := g(\text{embed_text}_i) \in \mathbb{R}^d,$$

where $g(\cdot)$ is a bi-encoder (e.g. off-the-shelf).

2.1.2 A2. Build ANN indices per retailer

For each retailer s , build an ANN index (e.g., FAISS HNSW/IVF) over vectors $\{e_i : i \in \mathcal{I}_s\}$.

2.1.3 A3. Retrieve top- K cross-retailer candidates

For each $i \in \mathcal{I}_{s_a}$ and each other retailer s_b :

$$\mathcal{C}_{s_b}(i) := \text{ANN}_{s_b}.\text{search}(e_i, K),$$

yielding K candidates $j \in \mathcal{I}_{s_b}$. This stage is *recall-oriented*; it does not decide identity.

2.2 Level B: Blocking / Hard Gating (Cheap Precision & Safety)

Goal. Eliminate obviously impossible candidates and reduce pathological mismatches before scoring.

For each candidate pair (i, j) from Level A:

2.2.1 B1. Own-brand filter (identity mode)

If

$$\text{own_brand}_i \vee \text{own_brand}_j = \text{True},$$

then either:

- **Drop** the pair (strict identity spec), or
- Route to a separate *substitution* pipeline (if product spec expands).

2.2.2 B2. Numeric agreement gate

If both size_i and size_j exist, require:

$$\Delta_{\text{size}}(i, j) \leq \tau_{\text{size, gate}}$$

(e.g., $\tau_{\text{size, gate}} \in [0.05, 0.15]$, this should be tuned). Pairs failing this are dropped.

2.2.3 B3. Category gate (optional)

If categories are reliable, enforce:

$$1_{\text{cat}}(i, j) = 1.$$

If category is noisy across retailers, keep it as a feature in Level C rather than a hard gate.

2.2.4 B4. Brand gate (optional)

If both branded and **brand** is high-quality, enforce $1_{\text{brand}}(i, j) = 1$; otherwise treat as a soft feature.

2.3 Level C: Ranker / Decision Model (Supervised Pair-wise Scoring)

Goal. Given the reduced candidate set after gating, decide whether a candidate pair is truly identical.

2.3.1 C1. Model

Train a supervised model $f_\theta(i, j)$ using pairwise features. Two common production choices:

- **GBDT** (LightGBM / XGBoost) on engineered pairwise features.
- **Cross-encoder** reranker (BERT-style) scoring (`embed_texti`, `embed_textj`) directly.

A pragmatic baseline is GBDT due to speed, interpretability, and good performance with structured features and missingness.

2.3.2 C2. Feature vector

Define the pairwise feature vector:

$$\phi(i, j) = [\text{dot}(i, j), \text{jac}(i, j), \text{tri}(i, j), \Delta_{\text{size}}(i, j), 1_{\text{unit}}(i, j), \\ 1_{\text{brand}}(i, j), 1_{\text{cat}}(i, j), \Delta_{\text{price}}(i, j), \text{missingness flags}, \dots].$$

Then the ranker is:

$$f_\theta(i, j) := \sigma(h_\theta(\phi(i, j)))$$

where h_θ is the GBDT logit output (or a neural score) and σ is a sigmoid.

2.4 Supervision: Bootstrapping Labels without GTIN

We assume there is no global ID for direct positives. We therefore construct *silver* labels and a small *gold* set to train f_θ above.

2.4.1 Silver Positive Labels (High Precision)

Silver positives are created by applying *stricter* versions of gating rules (and/or mutuality constraints) to obtain extremely reliable positives (high precision, lower recall).

A pair (i, j) is labeled as a silver positive if all hold:

1. **Not own-brand:** $\neg(\text{own_brand}_i \vee \text{own_brand}_j)$.
2. **Tight numeric agreement:** $\Delta_{\text{size}}(i, j) \leq \tau_{\text{size, pos}}$ (e.g., $\tau_{\text{size, pos}} = 0.02$).
3. **High lexical similarity:** $\text{tri}(i, j) \geq \tau_{\text{tri, pos}}$ or $\text{jac}(i, j) \geq \tau_{\text{jac, pos}}$.
4. **Mutual nearest neighbours:** $j \in \mathcal{C}_{s_b}(i)$ and $i \in \mathcal{C}_{s_a}(j)$ with small ranks (e.g., top-3, this should be tuned).
5. **Brand agreement when present:** if both `brand_norm` are non-empty, require $1_{\text{brand}}(i, j) = 1$.

2.4.2 Silver Negative Labels (High Precision)

Construct negatives from contradictions:

- **Numeric contradiction negatives:** same brand and category (when reliable) but $\Delta_{\text{size}}(i, j) \geq \tau_{\text{size, neg}}$ (e.g., 0.15 or higher).
- **Brand contradiction hard negatives:** very high lexical similarity but **brand** disagrees (and both are branded). These teach the model that superficial similarity is insufficient.
- **Category contradiction negatives:** if category is reliable, label pairs with strongly different categories as negative.

2.4.3 Gold Labels

Commission a small labeled set (e.g., 300–1000 pairs):

- Sample ambiguous pairs with model score in a mid-band (e.g., $0.4 \leq f_{\theta} \leq 0.9$).
- Sample a random audit of high-confidence positives for precision estimation.

Mostly for evaluation, calibration, and to prevent self-training drift.

2.4.4 LLM Labels

LLMs can assist as *weak supervision* after candidate retrieval:

1. Retrieve top- K candidates via Level A.
2. Apply Level B gating to remove absurd mismatches.
3. For remaining ambiguous pairs, prompt the LLM to output **SAME** / **DIFFERENT** / **UNCERTAIN** and extracted structured attributes (brand, size, pack).
4. Accept LLM **SAME** only if numeric constraints agree (e.g., Δ_{size} below threshold) to avoid numeric hallucinations.

Make sure to audit LLM outputs.

2.5 What Gets Trained at Each Level

Level A (bi-encoder).

- **Initially:** use an off-the-shelf sentence embedding model $g(\cdot)$.
- Optional fine-tuning downstream.

Level C (ranker). Train f_{θ} (GBDT or cross-encoder) on silver labels plus gold labels for calibration. The ranker is the primary precision driver.

2.6 Final Matching and Output Construction

For each anchor item $i \in \mathcal{I}_{s_a}$ and retailer s_b :

1. Retrieve candidates $\mathcal{C}_{s_b}(i)$ (Level A).
2. Apply gating (Level B).
3. Score remaining candidates with $f_\theta(i, j)$ (Level C) and select:

$$j^* = \arg \max_{j \in \mathcal{C}_{s_b}(i)} f_\theta(i, j).$$

4. Accept match if:
 - $f_\theta(i, j^*) \geq \tau_{\text{accept}}$, and
 - margin over runner-up exceeds τ_{margin} , and
 - numeric constraints (when available) are satisfied.

Otherwise, abstain (no match).